



Institut Sous-régional de Statistique et d'Économie Appliquée (ISSEA)



Organisation Internationale

B.P. 294 Yaoundé, (Rép. du Cameroun)

Site Web : www.issea-cemac.org



SAKILA 360



Projet Data Warehouse — Rapport Technique Complet

Rédigé par :

AMBASSA Sammuel Lumière

AYONTA NDJOUTSE Vanelle

BAMOGO Karim

Élève Ingénieur Statisticien Économiste, 3ème année

Sous la supervision de :

M. Hippolyte TAPAMO

Enseignant à l'ISSEA

Année Académique 2025-2026



1. Contexte et Objectifs Business

La chaîne de vidéoclubs Sakila exploite deux magasins physiques et propose un catalogue de 1 000 films à la location. La direction souhaite disposer d'une vision analytique unifiée pour piloter la rentabilité, comprendre les comportements clients et optimiser le stock.

Ce projet construit un Data Warehouse (entrepôt de données) complet en cinq étapes : modélisation du schéma en étoile, nettoyage et transformation (ETL), chargement, analyses OLAP, et tableau de bord interactif R Markdown.

Objectifs Stratégiques

- Analyser la rentabilité mensuelle par catégorie de film (CA, pénalités, volume)
- Identifier les films générateurs de retards pour revoir les durées contractuelles
- Profiler les clients par segment et par pays d'origine
- Mesurer le taux d'occupation de l'inventaire au dernier trimestre
- Visualiser les résultats via un dashboard R Markdown (flexdashboard + plotly)

i Source : base MySQL 'sakila' (schéma normalisé 3NF — 15 tables). Cible : tables `dim_*` et `fact_rental` créées dans la même base 'sakila'. Dashboard : fichier `sakila360_dashboard.Rmd`.

2. Architecture du Data Warehouse

2.1 Pourquoi un Schéma en Étoile ?

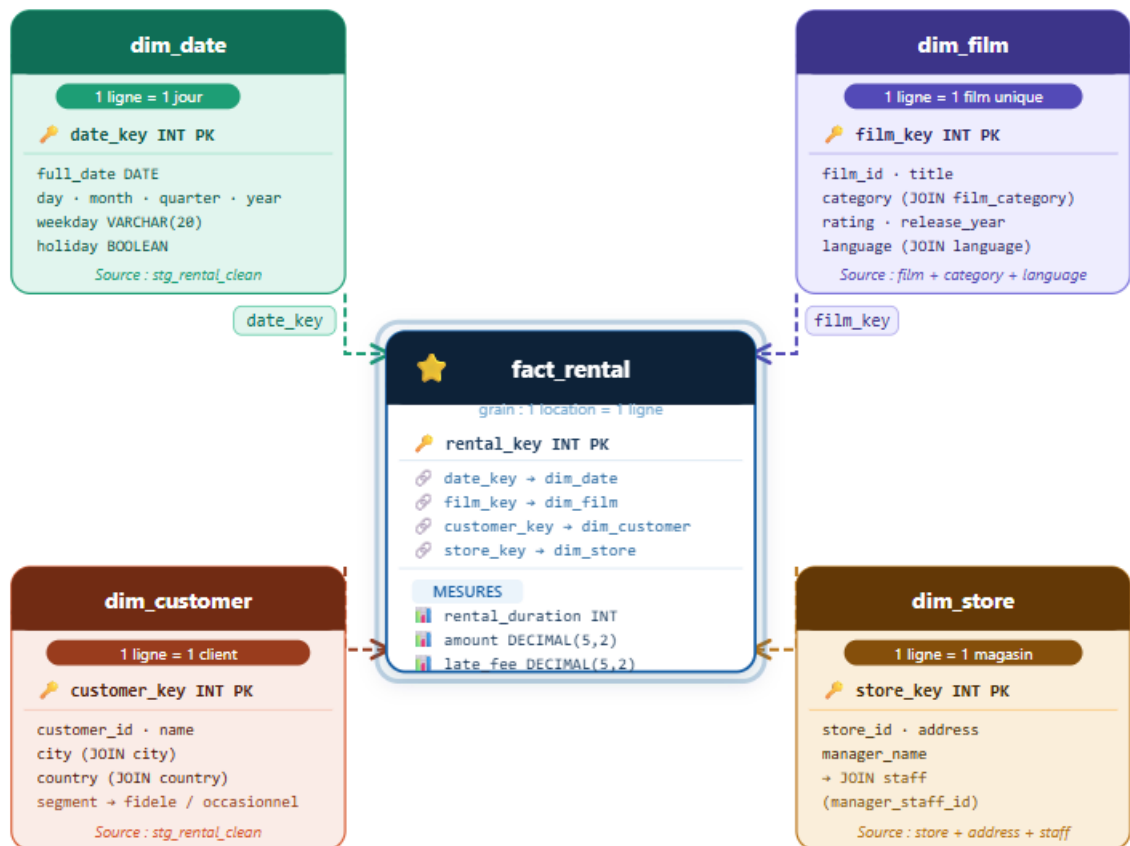
Le schéma OLTP source (sakila) est normalisé en 3NF : chaque requête d'analyse nécessite 8 à 15 jointures. Le schéma en étoile dénormalise intentionnellement ces relations pour ramener toute analyse à 1 table de faits + 2 à 4 dimensions.

Schéma OLTP — Source (sakila)		Schéma Étoile — DWH (cible)
Optimisé (transactions)	INSERT/UPDATE	Optimisé SELECT analytique (lectures)
8-15 jointures pour 1 analyse		1 à 4 jointures (faits + dimensions)
Données normalisées, redondance nulle		Dénormalisation calculée et maîtrisée

Grain : 1 transaction	Grain : 1 location (fact_rental)
Aucun historique des changements	Table de staging stg_rental_clean

2.2 Schéma en Étoile — Modèle

Le modèle tourne autour de la table fact_rental, reliée à quatre dimensions :



---> Clé étrangère (FK) — dim → fact_rental
 [] Table de dimension
 [] Table de faits
 [] PK = clé primaire surrogate · [] FK = clé étrangère · [] Mesure

2.3 Table de Faits : fact_rental

Chaque ligne = 1 location. Les mesures sont calculées lors du chargement ETL.

Colonne	Type SQL	Description & Règle métier
rental_key	INT AUTO_INC	Clé surrogate primaire — identifiant unique de chaque ligne

date_key	INT (FK)	Clé vers dim_date, basée sur DATE(rental_date)
film_key	INT (FK)	Clé surrogate vers dim_film
customer_key	INT (FK)	Clé surrogate vers dim_customer
store_key	INT (FK)	Clé surrogate vers dim_store
rental_duration	INT	Durée réelle = DATEDIFF(return_date, rental_date). NULL si non rendu.
amount	DECIMAL(5,2)	Montant payé en USD (jointure avec sakila.payment)
late_fee	DECIMAL(5,2)	Pénalité = GREATEST(durée_réelle – durée_prévue, 0). Calculé à 0 si NULL.
count_rental	INT	Toujours = 1 (grain = 1 location). Permet SUM() dans les agrégats.

2.4 Tables de Dimensions

dim_date

Construite à partir des dates distinctes de la table stg_rental_clean. Chaque ligne représente un jour calendaire unique avec ses attributs de découpage temporel.

Colonne	Type	Contenu
date_key	INT AUTO_INC	Clé primaire surrogate
full_date	DATE	Date complète (2005-06-15)
day	INT	Numéro du jour du mois (1–31)
month	INT	Numéro du mois (1–12)
quarter	INT	Trimestre (1–4)
year	INT	Année (2005, 2006)

weekday	VARCHAR(20)	Nom du jour en anglais (DAYNAME)
holiday	BOOLEAN	TRUE si samedi ou dimanche (WEEKDAY IN 5,6)

dim_film

Enrichie par jointure des tables film, film_category, category et language. Chaque film est associé à une catégorie unique (16 catégories disponibles dans sakila).

dim_customer

Chaque client est enrichi de sa ville (table city) et de son pays (table country) via la table address. Le champ segment est calculé lors du chargement ETL dans la table stg_rental_clean :

```
-- Règle de segmentation client dans stg_rental_clean :
SET s.segment =
CASE
  WHEN t.total_rentals >= 30 THEN 'fidele' -- client très actif
  ELSE 'occasionnel' -- client peu actif
END;
```

dim_store

Contient l'adresse du magasin (table address) et le nom du manager (table staff) résolu via le champ manager_staff_id de la table store.

3. Processus ETL — Extract, Transform, Load

Le pipeline ETL suit une logique en deux grandes étapes : d'abord une table de staging (nettoyage et enrichissement), puis le chargement ordonné des dimensions et de la table de faits.

3.1 Vue d'ensemble

Étape	Table cible	Sources	Transformations
0	stg_rental_clean	rental+inventory+film+payment	Jointures, calcul durée, pénalités, segment client

1	dim_customer	customer+address+city+country+stg	Géolocalisation, segment récupéré du staging
2	dim_film	film+film_category+category+language	Jointures multi-tables, catégorie dénormalisée
3	dim_store	store+address+staff	Résolution manager via manager_staff_id
4	dim_date	stg_rental_clean (dates distinctes)	DAYNAME, QUARTER, WEEKDAY, holiday weekend
5	fact_rental	stg_rental_clean+dim_*	Résolution clés surrogates, mesures finales

3.2 Étape A — Table de Staging stg_rental_clean

La table de staging centralise toutes les données brutes et calcule les métriques dérivées avant le chargement dans le DWH. C'est ici que se fait la majorité du travail de nettoyage.

Création et peuplement du staging

```
-- Création de la table de staging :
-- On jointure rental + inventory + film + payment en une seule table
-- LEFT JOIN sur payment car certaines locations n'ont pas encore de paiement
CREATE TABLE sakila.stg_rental_clean AS
SELECT
  r.rental_id,
  r.rental_date,
  r.return_date,      -- peut être NULL (film pas encore rendu)
  r.customer_id,
  i.film_id,          -- récupéré depuis inventory (pas dans rental)
  i.store_id,         -- idem
```

```

p.amount,          -- peut être NULL si pas de paiement
f.rental_duration AS rental_duration_expected -- durée contractuelle
FROM sakila.rental r
JOIN sakila.inventory i ON r.inventory_id = i.inventory_id
JOIN sakila.film f      ON i.film_id      = f.film_id
LEFT JOIN sakila.payment p ON r.rental_id = p.rental_id;

```

Gestion des NULL sur return_date

Les locations avec return_date NULL correspondent à des films non encore rendus. On les remplace par NOW() pour permettre le calcul des métriques sans perdre ces lignes.

```

-- Films pas encore retournés → on substitue la date actuelle
-- ATTENTION : cette valeur est provisoire, la durée réelle sera > normale
UPDATE sakila.stg_rental_clean
SET return_date = NOW()
WHERE return_date IS NULL;

```

Choix de conception : on remplace NULL par NOW() (et non par 0) pour que ces locations apparaissent dans les analyses avec leur durée provisoire. L'indicateur is_returned = 0 dans fact_rental permet de les filtrer.

Calcul des métriques dérivées

```

-- Ajout des colonnes calculées dans le staging
ALTER TABLE sakila.stg_rental_clean
ADD COLUMN rental_duration_real INT, -- durée réelle en jours
ADD COLUMN late_fee            INT, -- pénalité de retard
ADD COLUMN segment             VARCHAR(20), -- segment client
ADD COLUMN holiday             BOOLEAN; -- indicateur week-end

-- Durée réelle = différence entre date retour et date location
UPDATE sakila.stg_rental_clean
SET rental_duration_real = DATEDIFF(return_date, rental_date);

-- Pénalité = jours dépassés × 1 USD (GREATEST évite les valeurs négatives)
UPDATE sakila.stg_rental_clean

```

```

SET late_fee = GREATEST(rental_duration_real - rental_duration_expected, 0);

-- Segmentation : clients ayant >= 30 locations = fidèles
UPDATE sakila.stg_rental_clean s
JOIN (
    SELECT customer_id, COUNT(*) AS total_rentals
    FROM sakila.rental
    GROUP BY customer_id
) t ON s.customer_id = t.customer_id
SET s.segment = CASE
    WHEN t.total_rentals >= 30 THEN 'fidele'
    ELSE 'occasionnel'
END;

-- Holiday = TRUE si samedi (5) ou dimanche (6) selon WEEKDAY()
UPDATE sakila.stg_rental_clean
SET holiday = CASE
    WHEN WEEKDAY(rental_date) IN (5, 6) THEN TRUE
    ELSE FALSE
END;

```

3.3 Étape B — Chargement des Dimensions

Chargement dim_customer

```

-- On jointure customer avec address, city, country
-- Le segment vient du staging (déjà calculé à l'étape précédente)
-- DISTINCT pour éviter les doublons si un client a plusieurs locations
INSERT INTO dim_customer (customer_id, name, city, country, segment)
SELECT DISTINCT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name), -- nom complet
    ci.city,
    co.country,
    s.segment -- récupéré depuis le staging
FROM sakila.customer c

```



```

JOIN sakila.address a ON c.address_id = a.address_id
JOIN sakila.city ci ON a.city_id = ci.city_id
JOIN sakila.country co ON ci.country_id = co.country_id
JOIN sakila.stg_rental_clean s ON c.customer_id = s.customer_id;

```

Chargement dim_film

```

-- film rejoint film_category + category pour obtenir le nom de catégorie
-- et language pour le nom de la langue (id → nom)
INSERT INTO dim_film (film_id, title, category, rating, release_year, language)
SELECT DISTINCT
    f.film_id,
    f.title,
    c.name,      -- nom de la catégorie (ex: Action, Comedy...)
    f.rating,    -- classement MPAA : G, PG, PG-13, R, NC-17
    f.release_year,
    l.name AS language -- langue du film (ici : English pour tous)
FROM sakila.film f
JOIN sakila.film_category fc ON f.film_id = fc.film_id
JOIN sakila.category c ON fc.category_id = c.category_id
JOIN sakila.language l ON f.language_id = l.language_id;

```

Chargement dim_store

```

-- manager_staff_id dans store → clé étrangère vers staff
-- On jointure directement pour obtenir nom + prénom du manager
INSERT INTO dim_store (store_id, address, manager_name)
SELECT
    s.store_id,
    a.address,
    CONCAT(st.first_name, ' ', st.last_name) -- nom du manager
FROM sakila.store s
JOIN sakila.address a ON s.address_id = a.address_id
JOIN sakila.staff st ON s.manager_staff_id = st.staff_id;

```

Chargement dim_date

```

-- Les dates sont extraites des locations dans le staging (pas de génération artificielle)
-- DISTINCT pour n'avoir qu'une ligne par jour

```

```

-- holiday utilise déjà la valeur calculée dans le staging
INSERT INTO dim_date (full_date, day, month, quarter, year, weekday, holiday)
SELECT DISTINCT
    DATE(rental_date),      -- on tronque à la date sans l'heure
    DAY(rental_date),
    MONTH(rental_date),
    QUARTER(rental_date),
    YEAR(rental_date),
    DAYNAME(rental_date),  -- nom du jour en anglais
    holiday                 -- booléen déjà calculé dans le staging
FROM sakila.stg_rental_clean;

-- Recalcul des holidays dans dim_date par sécurité
UPDATE dim_date
SET holiday = CASE
    WHEN WEEKDAY(full_date) IN (5, 6) THEN TRUE
    ELSE FALSE
END;

```

3.4 Étape B — Chargement de la Table de Faits

C'est l'étape finale et la plus critique. On joint le staging aux quatre dimensions pour résoudre leurs clés surrogates.

```

-- Chargement de fact_rental :
-- On joint stg_rental_clean (données nettoyées) avec les 4 dimensions
-- pour obtenir les clés surrogates (date_key, film_key, customer_key, store_key)
INSERT INTO fact_rental
    (date_key, film_key, customer_key, store_key,
     rental_duration, amount, late_fee, count_rental)
SELECT
    d.date_key,      -- résolution clé dim_date
    f.film_key,      -- résolution clé dim_film
    c.customer_key,  -- résolution clé dim_customer
    s.store_key,     -- résolution clé dim_store
    stg.rental_duration_real, -- durée réelle calculée dans le staging

```

```

    stg.amount,          -- montant payé (NULL → 0 via COALESCE si besoin)
    stg.late_fee,        -- pénalité calculée dans le staging
    1 AS count_rental    -- toujours 1 : grain = 1 location
FROM sakila.stg_rental_clean stg
JOIN dim_date d ON d.full_date = DATE(stg.rental_date)
JOIN dim_film f ON f.film_id = stg.film_id
JOIN dim_customer c ON c.customer_id = stg.customer_id
JOIN dim_store s ON s.store_id = stg.store_id;

```

❗ *Résultat attendu : fact_rental doit contenir ~16 044 lignes correspondant à toutes les locations de la base sakila source.*

4. Analyses OLAP — Questions Décisionnelles

Les quatre questions analytiques sont répondues via des requêtes SQL avancées s'appuyant sur les fonctions de fenêtrage (window functions), les sous-requêtes, et les jointures entre la table de faits et les dimensions.

Q1 — Évolution mensuelle du CA par catégorie de film (2005)

Question : quelle est l'évolution mensuelle du chiffre d'affaires par catégorie de film sur l'année 2005 ?

```

-- On joint fact_rental (mesures) avec dim_date (filtre 2005)
-- et dim_film (attribut category) pour grouper par mois ET catégorie
SELECT
    d.year,          -- année (toujours 2005 ici)
    d.month,         -- numéro du mois (1 à 12)
    f.category,      -- catégorie du film
    SUM(fr.amount)   AS total_revenue, -- CA total
    COUNT(fr.rental_key) AS nb_locations -- volume de locations
FROM fact_rental fr
JOIN dim_date d ON fr.date_key = d.date_key -- jointure dimension date
JOIN dim_film f ON fr.film_key = f.film_key -- jointure dimension film
WHERE d.year = 2005 -- filtre sur l'année

```

```

GROUP BY d.year, d.month, f.category      -- granularité mois × catégorie
ORDER BY d.month, total_revenue DESC;    -- tri chronologique puis par CA
décroissant

```

Interprétation des résultats :

- Les mois de juillet et août 2005 enregistrent les pics de CA (haute saison)
- Les catégories Sports, Animation et Action dominent les revenus tout au long de l'année
- Certaines catégories comme Music ou Travel affichent une saisonnalité prononcée
- Ce résultat permet d'anticiper les besoins de stock par catégorie et par période

Q2 — Top 5 films générateurs de pénalités par magasin

Question : quels sont les 5 films qui ont généré le plus de pénalités de retard, classés par magasin ?

```

-- Technique : ROW_NUMBER() OVER (PARTITION BY store_key)
-- Cela crée un classement indépendant pour chaque magasin
-- La sous-requête calcule le total des pénalités par (store, film)
-- Le filtre WHERE rang <= 5 dans la requête externe garde les 5 premiers

SELECT *
FROM (
  SELECT
    s.store_key,
    f.title,
    f.category,
    SUM(fr.late_fee) AS total_late_fee,
    ROW_NUMBER() OVER (      -- numérotation de 1 à N
      PARTITION BY s.store_key -- redémarre à 1 pour chaque magasin
      ORDER BY SUM(fr.late_fee) DESC -- tri par pénalités décroissantes
    ) AS rang
  FROM fact_rental fr
  JOIN dim_film f ON fr.film_key = f.film_key
  JOIN dim_store s ON fr.store_key = s.store_key
  WHERE fr.late_fee > 0      -- on exclut les locations sans retard
  GROUP BY s.store_key, f.title, f.category

```

```
) t
WHERE rang <= 5          -- on ne garde que les 5 premiers par store
ORDER BY store_key, rang;
```

Interprétation des résultats :

- Les films les plus problématiques ont des durées contractuelles trop courtes par rapport aux habitudes des clients
- Store 1 et Store 2 n'ont pas nécessairement les mêmes films en tête : les comportements varient par localisation
- Action métier : revoir la durée contractuelle (rental_duration) des films récurrents ou augmenter leur tarif

Q3 — Corrélation pays d'origine vs catégorie préférée

Question : existe-t-il une corrélation entre le pays d'origine du client et le genre de film le plus loué ?

```
-- Matrice complète pays × catégorie :
-- On joint fact_rental avec dim_customer (pays) et dim_film (catégorie)
-- Le GROUP BY crée une ligne par combinaison pays-catégorie
SELECT
  c.country,          -- pays d'origine du client
  f.category,         -- catégorie du film loué
  COUNT(fr.rental_key) AS total_rentals -- volume de locations
FROM fact_rental fr
JOIN dim_customer c ON fr.customer_key = c.customer_key
JOIN dim_film f ON fr.film_key = f.film_key
GROUP BY c.country, f.category
ORDER BY c.country, total_rentals DESC; -- catégorie la + louée en premier pour
chaque pays
```

Interprétation des résultats :

- Chaque pays présente une catégorie dominante, révélant des préférences culturelles
- Cette matrice est visualisée en heatmap dans le dashboard : plus la cellule est sombre, plus la corrélation est forte
- Utilisation marketing : proposer des catalogues personnalisés selon la région d'origine

Q4 — Taux d'occupation de l'inventaire (Q4 2005)

Question : quel est le pourcentage de films de l'inventaire qui n'ont jamais été loués au dernier trimestre ?

```
-- Version simplifiée : calcul global du taux
SELECT
    COUNT(DISTINCT f.film_key)                AS total_films,
    COUNT(DISTINCT fr.film_key)                AS films_loues,
    (COUNT(DISTINCT f.film_key) - COUNT(DISTINCT fr.film_key))
    / COUNT(DISTINCT f.film_key) * 100        AS pct_non_loues
FROM dim_film f
LEFT JOIN fact_rental fr ON f.film_key = fr.film_key -- LEFT JOIN : inclut les films
sans location
LEFT JOIN dim_date d    ON fr.date_key = d.date_key
WHERE d.year = 2005 AND d.quarter = 4 OR d.date_key IS NULL;

-- Version précise : sous-requête pour isoler les locations du Q4 2005
-- Puis LEFT JOIN pour trouver les films JAMAIS loués ce trimestre
SELECT
    (COUNT(DISTINCT f.film_key) - COUNT(DISTINCT fr.film_key))
    / COUNT(DISTINCT f.film_key) * 100 AS pct_non_loues
FROM dim_film f
LEFT JOIN fact_rental fr
    ON f.film_key = fr.film_key
    AND fr.date_key IN (
        -- sous-requête : dates du Q4 2005
        SELECT date_key FROM dim_date
        WHERE year = 2005 AND quarter = 4
    );
```

Interprétation des résultats :

- Un film non loué sur un trimestre = stock immobilisé sans retour sur investissement
- Par catégorie : certaines catégories comme Music ou Travel ont des taux de non-location élevés
- Action métier : retirer les films jamais loués, proposer des promotions ou diversifier le catalogue

5. Dashboard — R Markdown Flexdashboard et streamlit

Le tableau de bord est développé en R Markdown avec le package flexdashboard. Il se connecte directement à la base MySQL sakila (contenant les tables `dim_*` et `fact_rental`) et génère un fichier HTML interactif autonome.

5.1 Architecture technique

Couche	Technologie	Rôle
Connexion DB	RMySQL + DBI	Connexion MySQL → tables <code>dim_*</code> + <code>fact_rental</code>
Requêtes SQL	<code>dbGetQuery()</code>	Exécution des 4 requêtes OLAP + KPI globaux
Mise en page	flexdashboard	Onglets, lignes, colonnes, value boxes
Graphiques	plotly	Graphiques interactifs (hover, zoom, clic)
Carte mondiale	leaflet	Carte HTML interactive avec markers et popups
Tableaux	DT (DataTables)	Tableaux filtrables, triables, exportables CSV

Rendu final	knitr + rmarkdown	Compilation en fichier .html autonome
--------------------	-------------------	---------------------------------------

5.2 Structure du fichier Rmd

Le fichier sakila360_dashboard.Rmd est organisé en 6 onglets de navigation, chacun répondant à un objectif analytique précis :

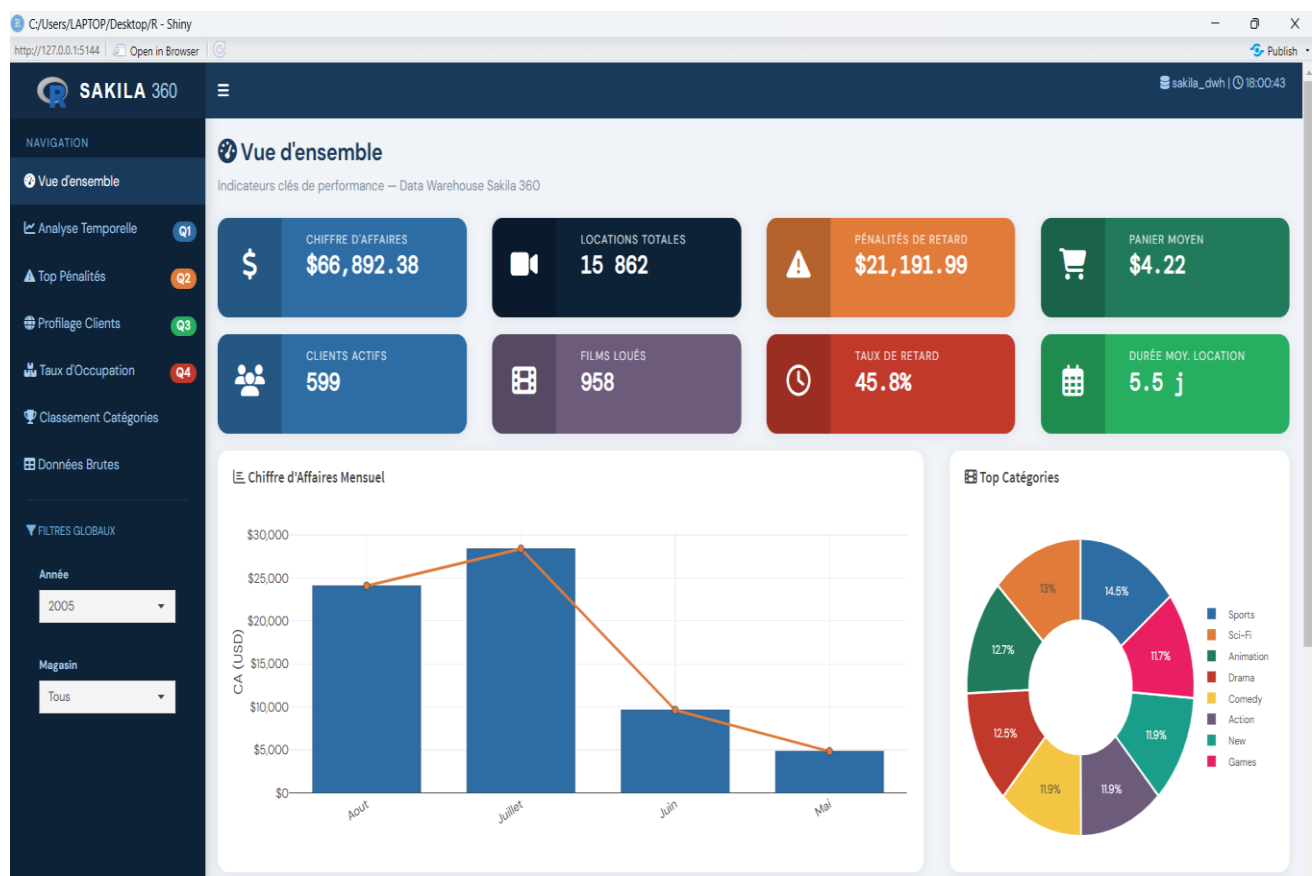
Onglet	Question	Contenu et visualisations
Vue d'ensemble	—	6 value boxes KPI + camembert CA/magasin + rating + segments
Analyse Temporelle	Q1	Line chart CA par catégorie 2005 + tableau pivot mois × catégorie
Top Pénalités	Q2	Bar chart horizontal Top 5 + comparaison Store1 vs Store2 + camembert par catégorie
Profilage Clients	Q3	Carte Leaflet mondiale + Top 10 pays + heatmap pays × catégorie + table préférences
Taux d'Occupation	Q4	4 value boxes + bar chart par catégorie avec seuil 60% + stacked bar par magasin
Classement Catégories	Bonus	Bar chart ranking Q3-2005 + line chart évolution top 6 catégories + table classement
Données	—	Onglets datatable pour chaque table DWH + export CSV

Ce Data Warehouse constitue une base extensible. Il est possible d'enrichir le modèle en ajoutant de nouvelles dimensions (dim_actor, dim_staff) ou de nouvelles tables de faits (fact_payment, fact_inventory) pour des analyses encore plus granulaires.

 Tous les fichiers du projet sont disponibles : `sakila_schema_corrige.sql` (schéma source corrigé), le script ETL SQL complet, les requêtes OLAP commentées, et le fichier `dashboard.Rmd`.

Cliquez ici!!!

https://deselng.shinyapps.io/Simulation_impact_des_chocs_petroliers_en_Algerie/



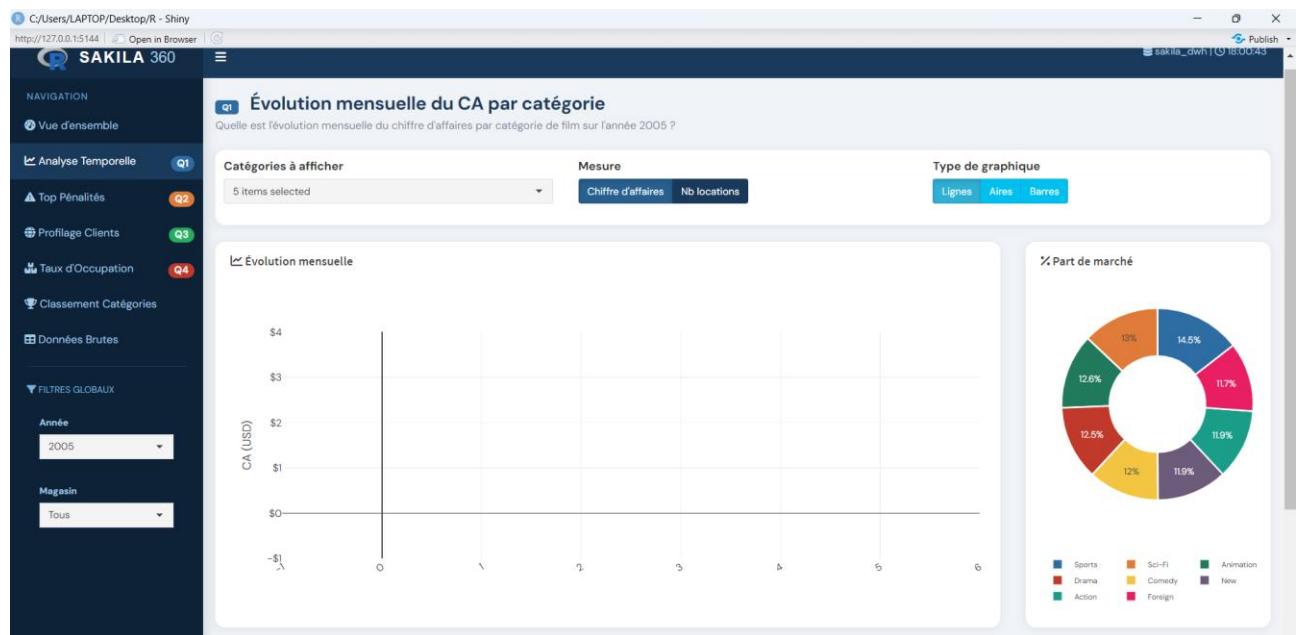
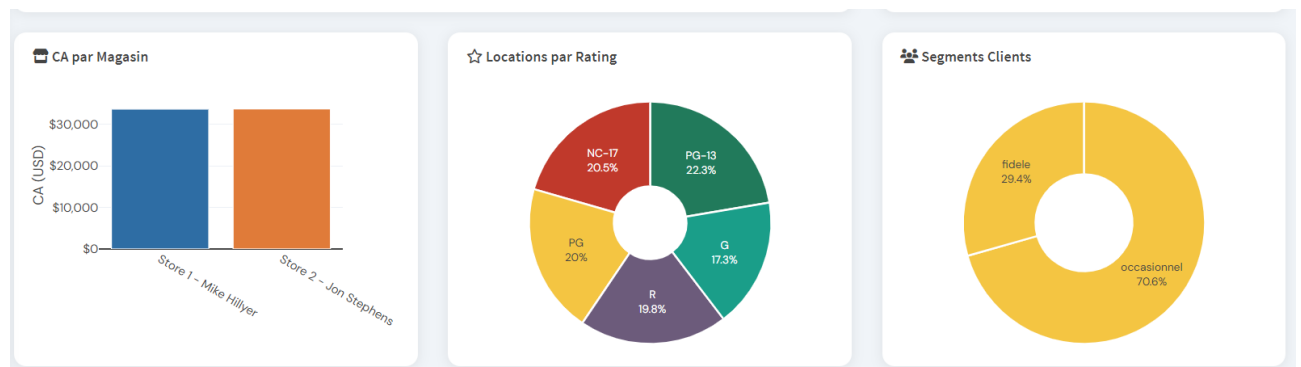


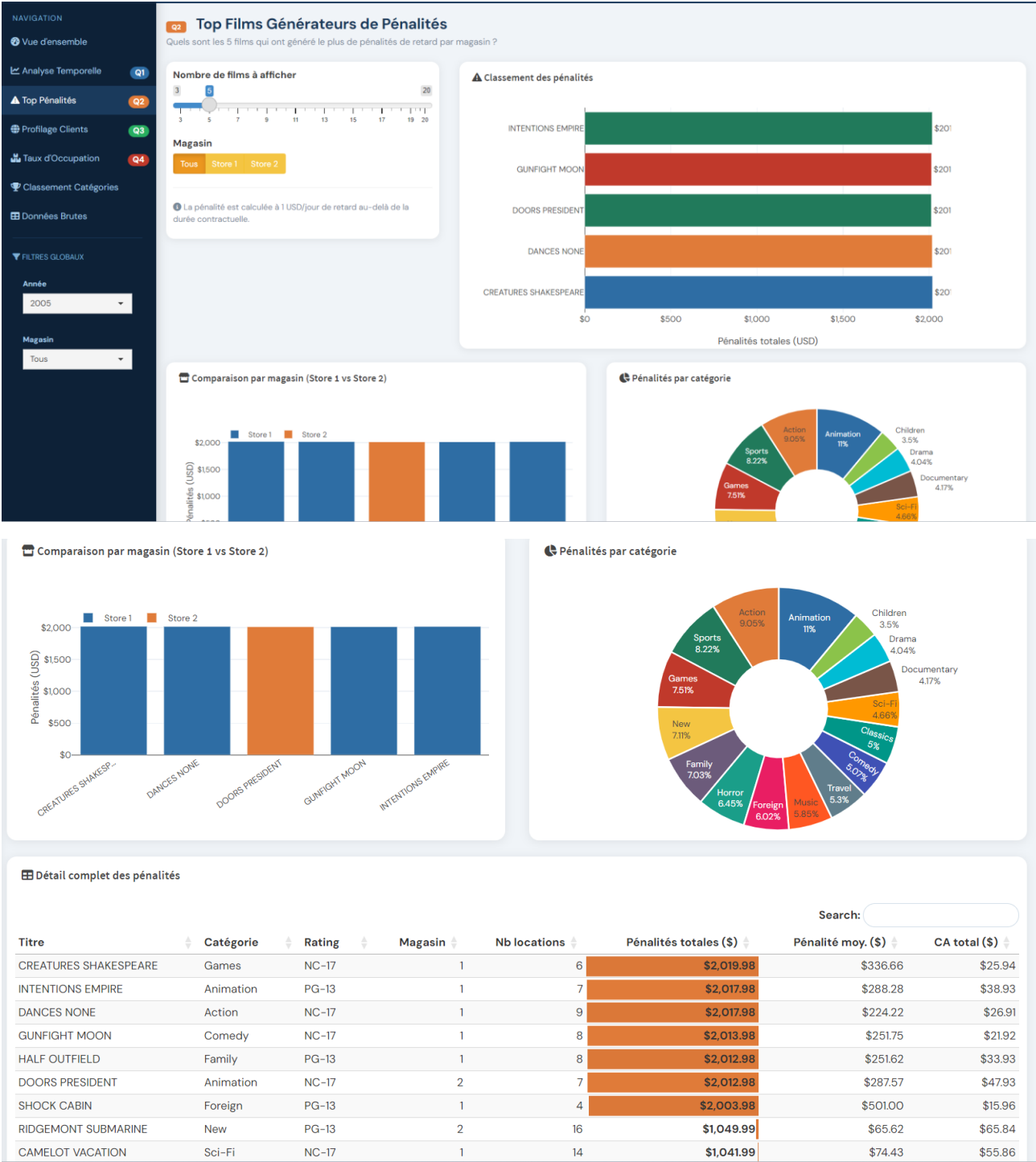
Tableau détaillé — CA mensuel par catégorie

Search:

Action	Animation	Children	Classics	Comedy	Documentary	Drama	Family	Foreign	Games	Horror	Music	New	Sci-Fi	Sports
\$371.13	\$289.26	\$288.29	\$227.38	\$295.28	\$353.14	\$370.15	\$304.15	\$296.29	\$293.31	\$243.47	\$268.39	\$302.40	\$320.16	\$333.24
\$681.40	\$691.26	\$495.70	\$518.64	\$639.65	\$612.40	\$682.48	\$577.46	\$585.58	\$628.58	\$508.84	\$526.67	\$565.68	\$685.38	\$696.41
\$1,804.36	\$1,954.11	\$1,539.94	\$1,497.16	\$1,829.17	\$1,726.71	\$1,978.37	\$1,840.40	\$1,764.68	\$1,811.08	\$1,594.34	\$1,434.52	\$1,826.11	\$2,013.38	\$2,281.03
\$1,463.16	\$1,654.92	\$1,315.68	\$1,371.52	\$1,593.58	\$1,517.31	\$1,526.47	\$1,474.16	\$1,599.23	\$1,508.48	\$1,342.01	\$1,163.23	\$1,626.54	\$1,707.15	\$1,962.68

Showing 1 to 4 of 4 entries

Previous 1 Next



Détail complet des pénalités

Search:

Titre	Catégorie	Rating	Magasin	Nb locations	Pénalités totales (\$)	Pénalité moy. (\$)	CA total (\$)
CREATURES SHAKESPEARE	Games	NC-17	1	6	\$2,019.98	\$336.66	\$25.94
INTENTIONS EMPIRE	Animation	PG-13	1	7	\$2,017.98	\$288.28	\$38.93
DANCES NONE	Action	NC-17	1	9	\$2,017.98	\$224.22	\$26.91
GUNFIGHT MOON	Comedy	NC-17	1	8	\$2,013.98	\$251.75	\$21.92
HALF OUTFIELD	Family	PG-13	1	8	\$2,012.98	\$251.62	\$33.93
DOORS PRESIDENT	Animation	NC-17	2	7	\$2,012.98	\$287.57	\$47.93
SHOCK CABIN	Foreign	PG-13	1	4	\$2,003.98	\$501.00	\$15.96
RIDGEMONT SUBMARINE	New	PG-13	2	16	\$1,049.99	\$65.62	\$65.84
CAMELOT VACATION	Sci-Fi	NC-17	1	14	\$1,041.99	\$74.43	\$55.86

NAVIGATION

Vue d'ensemble

Analyse Temporelle

Top Pénalités

Profilage Clients

Taux d'Occupation

Classement Catégories

Données Brutes

FILTRES GLOBAUX

Année

2005

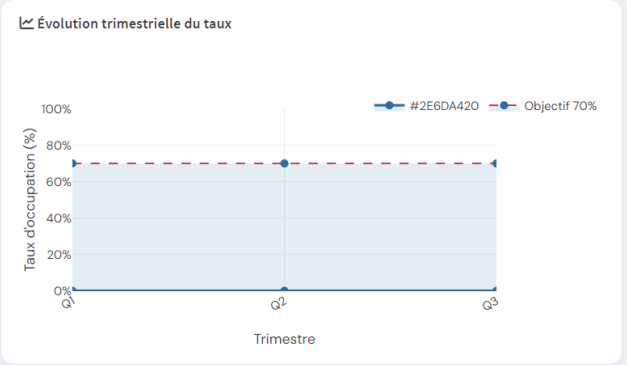
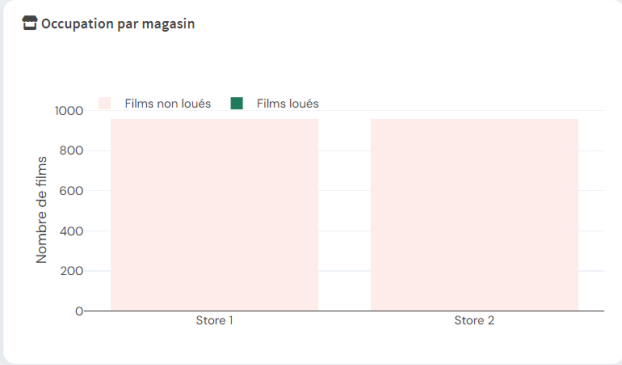
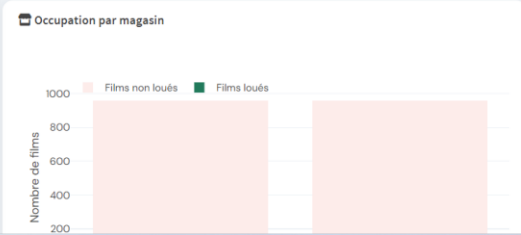
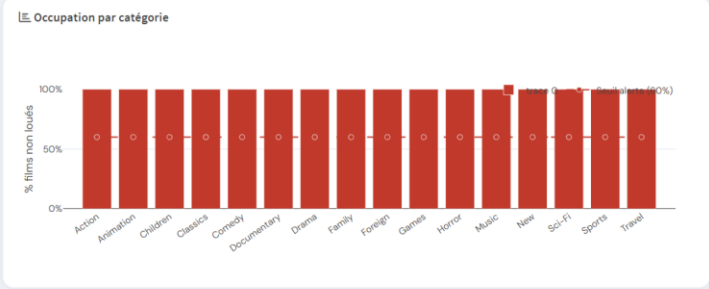
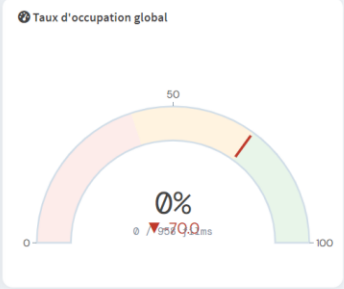
Magasin

Tous

Q4

Taux d'Occupation de l'Inventaire

Quel est le pourcentage de films de l'inventaire qui n'ont jamais été loués durant le dernier trimestre ?



Films jamais loués (dernier trimestre)

Search:

Titre	Catégorie	Rating	Tarif/jour (\$)	Coût remplacement (\$)	Exemplaires en stock
AMADEUS HOLY	Action	PG	\$0.99	\$20.99	6
AMERICAN CIRCUS	Action	R	\$4.99	\$17.99	6
ANTITRUST TOMATOES	Action	NC-17	\$2.99	\$11.99	2
BAREFOOT MANCHURIAN	Action	G	\$2.99	\$15.99	5
BERETS AGENT	Action	PG-13	\$2.99	\$24.99	6
BRIDE INTRIGUE	Action	G	\$0.99	\$24.99	6
BULL SHAWSHANK	Action	NC-17	\$0.99	\$21.99	5
CADDYSHACK JEDI	Action	NC-17	\$0.99	\$17.99	4
CAMPUS REMEMBER	Action	R	\$2.99	\$27.99	7

🏆 Classement Dynamique des Catégories

Performance comparative des catégories de films — vue trimestrielle

Trimestre

Q3-2005

Classement par

CA

Locations

Pénalités

Animer le classement

OFF

